THESIS REVIEWER · SECTION 3.2.2.3 · UPDATED AFTER THE INTERVIEWS

Reviewer: The SLM Algorithm (Algorithms 1–6)

A line-by-line guide to the algorithms in our manuscript, updated with the changes from the deaf students' interviews, with real runs through our code and the evidence for every number.

1. **How to read** the algorithm boxes
2. **The map**, and **what changed after the interviews**
3. **Algorithms 1 to 6**: the box, line by line, real runs, what to remember
4. **Follow the lines**: one conversation through every algorithm
5. **Where each number comes from** (for the adviser's questions)
6. **Cheat sheet** and **self-test** with answers

Part A · How to read an algorithm box

An algorithm box is a recipe: numbered steps the computer follows from top to bottom. Indented lines belong to the line above them (the inside of an **if** or a loop).

Symbol / word	Means	Example
←	"becomes": store the value on the right into the name on the left	*signs* ← [ ] : signs becomes an empty list
//	a comment for the reader; the computer ignores it	// 95 token IDs
if … **then** … **end if**	do the inside only when the condition is true	**if** *repair* **then** …
else if / **else**	otherwise, check this next / otherwise, do this	
for each x **in** L **do** … **end for**	repeat the inside once for every item of the list	for each sign received
for i ← 1 to n **do**	count i = 1, 2, … n and repeat each time	
while … **do** … **end while**	keep repeating while the condition stays true	while the system is on
repeat … **end repeat**	keep repeating until a **break** stops it	the microphone loop
procedure NAME(…) … **end procedure**	a small named algorithm that other algorithms call	UPDATE TOPIC(...)
return	give back the answer and finish	
break	jump out of the loop	stop recording
skip	ignore this item and go on to the next one	skip an unsure sign
∈ / ∉	is in / is not in	*w* ∈ LOOKUP TABLE
∪	put two groups together	SIGNS ∪ HELPER WORDS
∅ and [ ]	an empty set / an empty list	
==, =, ≠, <, ≥	equal / not equal / less than / at least	*confidence* < 0.6
AND, OR, NOT	both true / at least one true / the opposite	
italic	a variable: its value changes as the algorithm runs	*signs*, *negated*
CAPITALS	a fixed list or number, or the name of a procedure	TOPICS, FILL SUBJECT
x.y	the y part of x	memory.topic = the topic stored in the memory
[a] + b	put a in front of the list b	[person] + signs
<bos>, <eos>	special markers: beginning / end of a sentence	
// BLOCK n --- NAME	a label showing which box of the adviser's pipeline these lines are	// BLOCK 4 --- REDUCTION

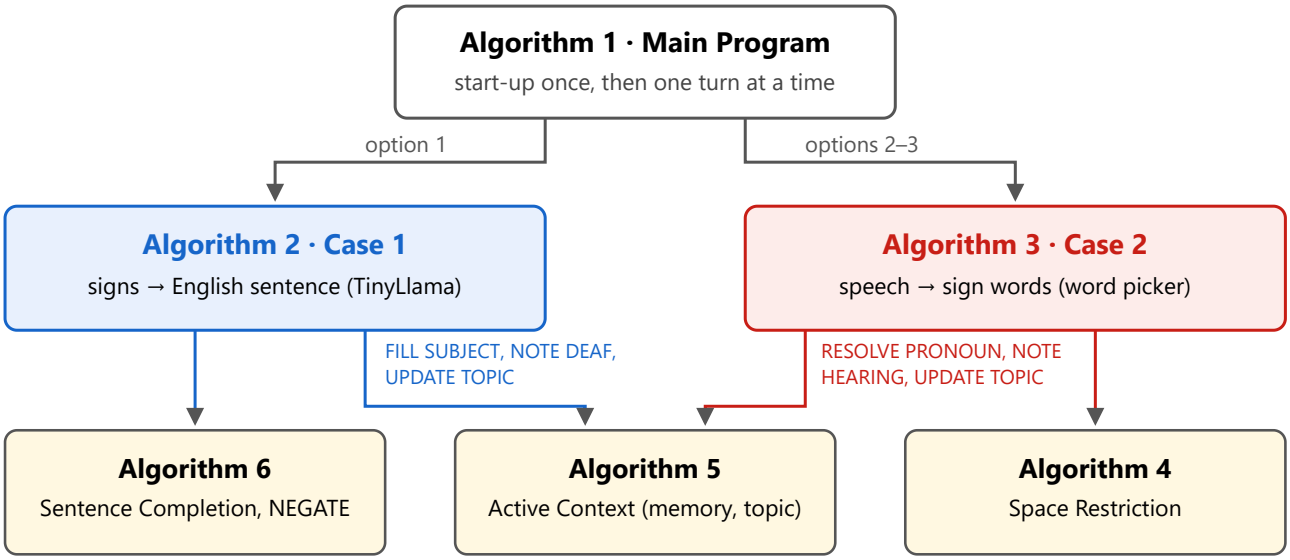
Words you will see everywhere

SIGN VOCABULARY	the 34 signs the camera can recognize
HELPER WORDS	57 words with no sign that people say (is, your, the, she, not …)
LOOKUP TABLE	signs + helper words = the 91 words the SLM knows
token ID	a word's number: 4 special markers + 91 words = 95 IDs
glosses	sign words in signing order, e.g. "where mother" (what the Deaf person reads)
refined text	the final text an algorithm gives back for the screen
confidence	how sure the sign recognizer is about a sign, from 0 to 1
NEGATIONS	words that flip the meaning: not, no, don't, never, can't …
negated	a yes/no flag: was NOT signed? (NOT is not one of the 34 signs, so it is kept as a flag)
repair rule	NOT + UNDERSTAND → ask the hearing person to say it again or write it
context memory	what the system remembers between turns: the person, the question words, whether the question was asked to "you", and the topic
topic	one of the 7 conversation contexts from the interviews (TOPICS)
OPTIONAL SIGNS	my, you: the only signs the training data leaves out, so the picker may drop them
QUESTION STARTERS	words a question starts with: is, are, do, does, can, will, what, where, why, how …

GREETING PHRASES	good morning / afternoon / evening / night: not in our vocabulary, so dropped
asked	a yes/no flag: did the hearing person ask a question (including a yes/no question)?
word picker / TinyLlama	our two trained models: Case 2 (hearing → Deaf) / Case 1 (Deaf → hearing)
Case 1 / Case 2	Case 1 = signer → non-signer (Deaf → hearing); Case 2 = non-signer → signer (hearing → Deaf)

Part B · The map

Algorithm 1 runs the program and decides who is talking. It hands each turn to one of the two pipelines, which call the three shared procedures at the bottom.



Where each box of the adviser's pipeline is

Pipeline box	Case 1 · Deaf → hearing	Case 2 · hearing → Deaf
Translated / Transcribed Word	Alg 2, Block 1 (lines 1–17)	—
Tokenize	Alg 2, Block 2 (line 19)	Alg 3, Block 5 (line 45)
Embedding	Alg 2, Block 3 (line 21)	Alg 3, Block 6 (line 47)
Active Context	Alg 2, Block 4 (lines 23–30) → Alg 5	Alg 3, Block 9 (lines 56–59) → Alg 5
Sentence Completion	Alg 2, Block 5 (lines 32–46) → Alg 6	—
Voice Recording	—	Alg 3, Block 1 (lines 1–10)
Speech to Text	—	Alg 3, Block 2 (lines 11–12)
Normalization	—	Alg 3, Block 3 (lines 17–30)
Reduction	—	Alg 3, Block 4 (lines 32–43)
Contextualize	—	Alg 3, Block 7 (lines 49–51)
Space Restriction	—	Alg 3, Block 8 (lines 53–54) → Alg 4
Transcribed Words	—	Alg 3, Block 10 (line 61)
Display	Alg 2, Block 6 (line 48) → Alg 6 → Alg 1 line 23	Alg 3, Block 11 (line 63) → Alg 1 line 28

What changed after the interviews, the adviser's consultation and live testing

Change	Where	Why
Unsure signs are dropped	`Alg 1 line 13; Alg 2 lines 3–5`	Students mix in gesture and home sign (5 of 5 groups): "unsure signs never reach the language model".
NOT is never thrown away	`Alg 2 lines 6–7, 13–14, 41–43; Alg 6 lines 19–31`	Students use NOT in 5 of 5 groups. Before: NOT UNDERSTAND came out as "i understand".
Repair rule	`Alg 2 lines 23, 32–33`	Students say they do not understand in 4 of 5 groups (High priority).
Seven topics	`Alg 1 line 10; Alg 2 line 24; Alg 3 lines 58–59; Alg 5 lines 41–63`	Every topic appeared in 4 or 5 groups (High priority).
Other contexts leak; switch at half (adviser)	`Alg 5 lines 41–63`	Adviser: the other contexts must not drop to 0, they leak in; the whole context changes only when another context covers at least half of the turn (e.g., 9 of 16). Before: one topic, all others 0, and a single trigger sign kept the old topic.
Person filled in only when the signs are about a person	`Alg 5 lines 20–33`	"Do not fill in … output only what was recognized". Before: "where water" → "mother where water".
Recording stops when the speaker stops	`Alg 1 lines 14–15; Alg 3 lines 1–10`	The device must be faster than typing on a phone (phone used in 3 of 5 groups).
Signs keep the order they were spoken in; no message sign is dropped	`Alg 4 lines 1–19`	Live test: "Are you ready to study for class today?" came out as "class you today study good"; 604 of the 608 training targets keep spoken order; the data drops only my and you.
Good morning and similar phrases are dropped	`Alg 3 line 18`	Live test: "good morning" was shown as the sign GOOD; the phrase is not in our vocabulary.
A "not" goes with its word	`Alg 3 lines 33–39; Alg 4 lines 6–8`	Live test: "Please do not worry, it is fine" reached the Deaf reader as "not please fine".
Punctuation only for the hearing reader	`Alg 2 line 48; Alg 6 lines 32–39; Alg 3 line 61`	Group decision from testing: the Deaf reader's screen stays plain sign words; the hearing reader gets "?" or ".".
A reply to a yes/no question is a statement	`Alg 3 lines 17, 57; Alg 5 lines 10–12`	Live test: after "Is your mother here?", MOTHER HOME came out as the question "is mother home".

Algorithm 1 Main Program

> **In one sentence:** Set everything up once, then wait for someone to talk, send the turn to the right pipeline, and show the text on the screen of the person who should read it.

> **New in this version:** the memory now also holds the **topic** (line 9); the **7 topics** are loaded (line 10); and three new numbers are set: the **confidence threshold**, the **sample rate** and the **recording limits** (lines 13–15). Case 1 now receives each sign **with its confidence** (line 20).

Algorithm 1: Two-Way Sign-Speech Communication System (Main Program)

Input: mode selection; signs with confidence (Case 1) or speech / typed text (Case 2)
Output: context-aware text on the display facing the intended reader

```
1   Initialize:
2       SIGN VOCABULARY ← 34 confirmed signs;
3       LOOKUP TABLE ← SIGN VOCABULARY ∪ HELPER WORDS;
4       tokenizer ← buildTokenizer(LOOKUP TABLE); // 91 words, 95 token IDs
5       wordPicker ← loadModel(trained word picker); // encoder, Case 2
6       tinyLlama ← loadModel(trained TinyLlama); // decoder, Case 1
7       set wordPicker and tinyLlama to evaluation mode;
8       micReady ← isAvailable(microphone, speech-to-text model);
9       context memory ← empty; // person, questions, asked to you, topic
10      TOPICS ← 7 topics with their trigger signs; // from the interviews
11      Define MAX SEQUENCE LENGTH ← 16;
12      Define MAX OUTPUT LENGTH ← 16;
13      Define SIGN CONFIDENCE THRESHOLD ← 0.6;
14      Define SAMPLE RATE ← 16,000; // the speech model's sample rate
15      Define MAX RECORD SECONDS ← 8, NO SPEECH SECONDS ← 5;
16
17  while system is active do
18      mode ← SELECT MODE(); // option 1 → CASE 1; options 2–3 → CASE 2
19      if mode == CASE 1 (Signer → Non-signer) then
20          signs ← receiveSigns(); // each sign with its confidence
21          // Algorithm 2
22          refined text ← SIGN TO SENTENCE(signs, context memory);
23          display(refined text, screen facing the non-signer);
24      else if mode == CASE 2 (Non-signer → Signer) then
25          typed ← typed sentence, or none when using the microphone;
26          // Algorithm 3
27          refined text ← SPEECH TO GLOSS(typed, context memory);
28          display(refined text, screen facing the deaf-mute individual);
29      else if mode == RESET then
30          context memory ← empty;
31      else if mode == QUIT then
32          break;
33      end if
34  end while
```

Line by line

Lines	What happens (plain words)	Why
1	**Initialize**: lines 2–15 happen **once**, when the program starts.	Loading is slow; doing it once makes every turn fast.
2–3	34 signs; the lookup table = 34 signs + 57 helper words = 91 words.	All the words the SLM knows.
4	Tokenizer: every word gets a number (4 special markers + 91 words = 95 IDs).	Models compute with numbers.
5–7	Load the two trained models and switch off training randomness.	Same input → same output.
8	Check that the microphone and the Vosk speech model are installed.	If not, typing still works.
9	Empty memory: no person, no question, not asked to "you", **topic neutral**, every topic weight 0.	A new conversation.
10	**TOPICS**: help and repair, school and learning, whereabouts and plans, greeting, courtesy, home and family, feelings, each with its trigger signs.	The 7 contexts found in the interviews.
11–12	At most 16 words in and 16 words out.	The models have 16 positions.

Lines	What happens (plain words)	Why
13	**Confidence threshold 0.6**: a sign the recognizer is less sure of is dropped.	Never guess. (0.6 is a placeholder until it is calibrated, see Part E.)
14	Record sound at **16,000 samples per second**.	The speech model was built for 16 kHz (Part E).
15	A spoken turn may last at most **8 s**; if nobody speaks for **5 s**, give up.	Part E explains both numbers.
17	**while**: repeat lines 18–33 for every turn.	
18	Read the mode: in the demo 1, 2, 3, r or q.	Decides who is talking.
19–23	**Case 1**: receive the signs with their confidence, run Algorithm 2, show the sentence on the non-signer's screen.	Deaf → hearing.
24–28	**Case 2**: typed sentence, or the microphone; run Algorithm 3; show the sign words on the deaf-mute individual's screen.	Hearing → Deaf.
29–30	RESET empties the memory, including the topic.	A new conversation.
31–34	QUIT leaves the loop; end markers.	

Try it: real runs through our code

A real 8-turn conversation (the memory and topic carry over between turns):

Turn	Case	Input	Runs	Screen shows	Topic after
1	2	Hearing person types: “Where is your mom?”	Alg 3	**where mother**	whereabouts and plans
2	1	Deaf person signs: “home”	Alg 2	**mother is home.**	whereabouts and plans
3	2	Hearing person types: “Is she okay?”	Alg 3	**how mother**	home and family
4	1	Deaf person signs: “fine”	Alg 2	**mother is fine.**	feelings
5	2	Hearing person types: “I do not understand, can you say that again?”	Alg 3	**not understand again**	help and repair
6	1	Deaf person signs: “please again”	Alg 2	**please say it again.**	help and repair
7	2	Hearing person types: “What time is class tomorrow?”	Alg 3	**what time class tomorrow**	help and repair
8	1	Deaf person signs: “school tomorrow”	Alg 2	**i will go to school tomorrow.**	whereabouts and plans

> **Remember**
>
> - Lines 1–15 run **once**; lines 17–34 run **once per turn**.
> - The memory now holds four things: **person, question words, asked to you, topic** (with a weight for each of the 7 contexts).
> - Both cases end with **text** on a screen.

Algorithm 2 Sign-to-Sentence Pipeline (Case 1)

In one sentence: Turn the signs the Deaf person made into a complete English sentence for the hearing person.

New in this version: unsure signs are dropped (lines 4–5); NOT is kept as a flag (lines 6–7, 13–14); the **repair rule** (lines 23, 32–33); the topic is updated (line 24); a person is added only when the signs are about a person (line 26); NOT is put into the sentence (lines 41–43); the sentence gets **"?" or "."** for the hearing reader (line 48).

Algorithm 2: Sign-to-Sentence Pipeline (Case 1: Signer → Non-signer)

Input: *signs* (recognized sign words with their confidence), context memory
Output: *refined text* (English sentence for the display facing the non-signer)

```
1   // BLOCK 1 --- TRANSCRIBED WORD
2   kept ← [ ]; negated ← false;
3   for each (token, confidence) in signs do
4       if confidence < SIGN CONFIDENCE THRESHOLD then
5           skip token; // unsure sign: never guessed
6       else if token ∈ NEGATIONS then
7           negated ← true; // NOT is kept as a flag, never dropped
8       else if normalizeWord(token) ∈ SIGN VOCABULARY then
9           append normalizeWord(token) to kept; // e.g., mama → mother
10      end if
11  end for
12  signs ← first MAX SEQUENCE LENGTH words of kept;
13  if signs is empty AND negated then
14      return "no"; // only NOT was signed
15  else if signs is empty then
16      return "";
17  end if
18  // BLOCK 2 --- TOKENIZE
19  ids ← tokenizer.encode(signs);
20  // BLOCK 3 --- EMBEDDING
21  M ← tinyLlama.embedSource(ids); // one 64-value vector per sign, no position
22  // BLOCK 4 --- ACTIVE CONTEXT (Algorithm 5)
23  repair ← negated AND "understand" ∈ signs;
24  UPDATE TOPIC(signs, repair, context memory); // NOT counts if negated
25  if NOT repair then
26      (signs, person) ← FILL SUBJECT(signs, context memory);
27      if person ≠ none then
28          M ← concatenate(tinyLlama.embedSource(person), M);
29      end if
30  end if
31  // BLOCK 5 --- SENTENCE COMPLETION (Algorithm 6)
32  if repair then
33      refined text ← REPAIR SENTENCE; // ask to say it again or write it
34  else
35      refined text ← GENERATE SENTENCE(M);
36      if context memory.last was question then
37          refined text ← toDeclarative(refined text); // question form → statement
38      end if
39      if some sign in signs is missing from refined text then
40          refined text ← join(signs); // safety net: show the signs
41          if negated then refined text ← "not " + refined text;
42      else if negated then
43          refined text ← NEGATE(refined text); // i am fine → i am not fine
44      end if
45  end if
46  NOTE DEAF(signs, refined text, context memory);
47  // BLOCK 6 --- DISPLAY
48  return ADD PUNCTUATION(refined text); // Algorithm 6
```

Line by line

Lines	What happens (plain words)	Why
1–2	**Block 1 · Transcribed Word.** Start an empty list *kept*; no NOT seen yet.	
3	Look at every sign the recognizer sent, together with its **confidence** (0 to 1).	

Lines	What happens (plain words)	Why
4–5	If the recognizer is **less than 0.6 sure**, skip the sign.	Students mix in gesture and home sign; guessing would mislead the hearing person.
6–7	If it is a negation (not, no, don't …), set *negated* and go on.	NOT is not one of the 34 signs; dropping it would flip the meaning.
8–9	Otherwise turn it into its sign name (mama → mother) and keep it if it is one of the 34 signs.	TinyLlama only learned the 34 sign names.
12	Keep at most 16 signs.	The model's limit.
13–14	If only NOT was signed, answer "no".	NOT alone still means no.
15–16	If nothing is left, return empty text.	Nothing to translate.
19	**Block 2 · Tokenize.** Each sign → its ID (home → 36).	
21	**Block 3 · Embedding.** Each ID → 64 numbers. **No position**.	The order signs arrive in does not matter.
23	**Block 4 · Active Context.** *repair* is true when NOT and UNDERSTAND were both signed.	The student is saying "I don't understand".
24	UPDATE TOPIC (Algorithm 5, lines 41–63). NOT counts as a help-and-repair sign.	Keeps track of the topic.
25–30	If it is not a repair: FILL SUBJECT (Algorithm 5, lines 20–33) may add the remembered person, and that person's 64 numbers go in front of *M*.	"home" after "where is your mom?" means mother.
32–33	**Block 5 · Sentence Completion.** Repair → the fixed sentence "i do not understand please say it again or write it".	The interviews ask for exactly this (High priority).
35	Otherwise GENERATE SENTENCE (Algorithm 6, lines 1–18) writes the sentence word by word.	
36–38	If the hearing person just asked a question, turn the question form into a statement.	"is mother home" → "mother is home".
39–41	Safety net: if any sign is missing, show the signs themselves (with "not" first if negated).	Never a fluent sentence about something else.
42–43	Otherwise, if NOT was signed, NEGATE (Algorithm 6, lines 19–31) puts it into the sentence.	"i am fine" → "i am not fine".
46	NOTE DEAF (Algorithm 5, lines 34–40) updates the memory.	
48	**Block 6 · Display.** ADD PUNCTUATION (Algorithm 6, lines 32–39) ends the sentence with "?" or "."; Algorithm 1 line 23 shows it.	Punctuation is for the hearing reader only.

Try it: real runs through our code

Unsure signs (lines 4–5). Typed as sign:confidence in the demo:

Signs with confidence	Block 1 result	Screen
mother:0.9 home:0.4	mother [unsure, dropped: home (0.40)]	**this is mother.**
mother:0.95 home:0.85	mother home	**is mother home?**
home:0.2	(no signs) [unsure, dropped: home (0.20)]	**(nothing)**

Repair rule (lines 23, 32–33):

Signs	Block 1 result	Screen
not understand	NOT understand	**i do not understand please say it again or write it.**
help please not understand	NOT help please understand	**i do not understand please say it again or write it.**

NOT kept and put into the sentence (lines 6–7, 13–14, 41–43):

Signs	Block 1 result	Screen
not fine	NOT fine	**i am not fine.**
not water	NOT water	**i do not want water.**
not eat	NOT eat	**i am not eating.**
not school tomorrow	NOT school tomorrow	**i will not go to school tomorrow.**

Signs	Block 1 result	Screen
not mother home	NOT mother home	**mother is not home.**
not	NOT	**no.**
not fine ← after “how is your father”	NOT fine	**father is not fine.**

The same sign, different memory (Block 4):

Situation	Active Context did	Screen
home (memory empty)	no person remembered yet; topic neutral → whereabouts and plans (100% of the context words)	**i am home.**
mama home	a person was signed; topic neutral → home and family (100% of the context words)	**is mother home?**
home, right after “Where is your mom?”	added 'mother' from earlier; topic stays whereabouts and plans	**mother is home.**

> **Remember**
> - Block 1 drops **unsure** signs and keeps **NOT** as a flag.
> - NOT + UNDERSTAND → the **repair** sentence, whatever the topic.
> - A person is added only when the signs are **about a person** (Algorithm 5).
> - A signed NOT always reaches the hearing person (NEGATE or the safety net).

Algorithm 3 Speech-to-Gloss Pipeline (Case 2)

In one sentence: Shrink what the hearing person said into the sign words the Deaf person knows, in signing order.

New in this version: the microphone **stops when the speaker stops** instead of always recording 5 s (lines 1–10); a **question**, yes/no questions too, is noted (line 17); **good morning** and similar phrases are dropped (line 18); a **"not"**

goes with its word (lines 33–39); the **topic** is updated (lines 58–59); the Deaf reader's screen has **no punctuation** (line 61).

Algorithm 3: Speech-to-Gloss Pipeline (Case 2: Non-signer → Signer)

Input: *typed* (typed sentence, or none to use the microphone), context memory
Output: *refined text* (sign-word glosses for the display facing the deaf-mute individual)

```
1   if typed = none then
2       // BLOCK 1 --- VOICE RECORDING (stops when the speaker stops)
3       audio ← [ ]; heard ← false;
4       repeat
5           append readMicrophone(0.25 s, SAMPLE RATE) to audio;
6           if vosk.endOfSpeech(audio) AND heard then break; // ≈0.9 s silence
7           if vosk.hearsWords(audio) then heard ← true;
8           if MAX RECORD SECONDS reached then break;
9           if no words heard after NO SPEECH SECONDS then break;
10      end repeat
11      // BLOCK 2 --- SPEECH TO TEXT
12      transcript ← vosk.transcribe(audio); // pre-trained, offline
13  else
14      transcript ← typed; // typed input skips Blocks 1–2
15  end if
16  // BLOCK 3 --- NORMALIZATION
17  asked ← transcript has "?" OR starts with a QUESTION STARTER;
18  text ← lowercase(transcript) without GREETING PHRASES; // good morning
19  text ← removePunctuation(text);
20  words ← [ ];
21  for each w in split(text) do
22      if w ∈ CONTRACTIONS then
23          w ← expand(w); // don't → do not
24      else if w is a possessive of a LOOKUP TABLE word then
25          w ← baseWord(w); // dad's → father
26      else if w ∈ SYNONYMS AND w ∉ TRAINED WORDS then
27          w ← SYNONYMS[w]; // mom → mother
28      end if
29      append w to words;
30  end for
31  // BLOCK 4 --- REDUCTION
32  kept ← [ ];
33  for each w in words do
34      if w ∈ NEGATIONS AND it negates a word ∉ LOOKUP TABLE then
35          skip w; // "do not worry": the not goes with worry
36      else if w ∈ LOOKUP TABLE then
37          append w to kept;
38      end if
39  end for
40  kept ← first MAX SEQUENCE LENGTH words of kept;
41  if kept is empty then
42      return "";
43  end if
44  // BLOCK 5 --- TOKENIZE
45  ids ← tokenizer.encode(kept);
46  // BLOCK 6 --- EMBEDDING
47  X ← wordPicker.embed(ids); // word vector + position vector, n × 64
48  // BLOCK 7 --- CONTEXTUALIZE
49  for layer ← 1 to 2 do
50      X ← TransformerBlock(layer, X); // self-attention + feed-forward
51  end for
52  // BLOCK 8 --- SPACE RESTRICTION (Algorithm 4)
53  (tagScores, questionScores) ← wordPicker.heads(X);
54  glosses ← RESTRICT TO SIGNS(tagScores, questionScores, kept);
55  // BLOCK 9 --- ACTIVE CONTEXT (Algorithm 5)
56  glosses ← RESOLVE PRONOUN(words, glosses, context memory);
57  NOTE HEARING(words, glosses, asked, context memory);
58  repair ← some w ∈ NEGATIONS AND "understand" ∈ words;
59  UPDATE TOPIC(words ∪ glosses, repair, context memory);
60  // BLOCK 10 --- TRANSCRIBED WORDS
61  refined text ← join(glosses); // plain sign words, no punctuation
62  // BLOCK 11 --- DISPLAY
63  return refined text; // shown on the display facing the deaf-mute individual
```

Line by line

Lines	What happens (plain words)	Why
1	If nothing was typed, use the microphone.	
2–3	**Block 1 · Voice Recording.** Start with no audio; no words heard yet.	
4–5	**repeat**: read a quarter second of sound (16,000 samples per second).	Small pieces let us stop quickly.
6	Stop when Vosk detects the **end of speech** after words were heard (≈0.9 s of silence).	No need to wait a fixed 5 s (Part E).
7	Once Vosk hears any word, remember that someone is speaking.	
8	Stop at 8 s at most.	A sentence of 16 words fits (Part E).
9	If no words were heard after 5 s, stop.	Nobody is talking (Part E).
11–12	**Block 2 · Speech to Text.** Vosk writes down the words.	Our models read words, not sound.
13–14	If it was typed, use the typed sentence and skip Blocks 1–2.	Option 2 of the demo.
17	**Block 3 · Normalization.** *asked*: did the hearing person ask a question? ("?", or it starts with is, are, do, can, what …)	So a reply to "Is your mother here?" is written as a statement.
18–19	Drop greeting phrases outside our vocabulary (good morning …), then remove punctuation.	"good morning" must not show the sign GOOD.
20–30	Lowercase words: don't → do not, dad's → father, mom → mother.	Words must arrive in the form the model learned.
32–39	**Block 4 · Reduction.** Keep only lookup-table words. A "not" whose word is not in the lookup table goes with it.	"do not worry, it is fine" must not become "not … fine".
40–43	At most 16 words; nothing left → empty text.	The models' limit.
45–47	**Blocks 5–6.** Words → IDs → 64 numbers each (word meaning + position).	In English, order matters.
49–51	**Block 7 · Contextualize.** Two transformer blocks: every word looks at every other word.	
53–54	**Block 8 · Space Restriction.** Scores, then RESTRICT TO SIGNS (Algorithm 4).	Only signs reach the screen.
56–57	**Block 9 · Active Context.** he/she → remembered person; remember the person, the question words and *asked*.	
58–59	If the hearing person said NOT + UNDERSTAND, it is a repair; UPDATE TOPIC with the words and glosses.	The hearing side also moves the topic.
61–63	**Blocks 10–11.** Join the glosses as plain sign words; Algorithm 1 line 28 shows them.	No punctuation on the Deaf reader's screen.

Try it: real runs through our code

Where is your mom?, block by block:

Block (lines)	Real value
Blocks 1–2 (1–14)	typed, so the microphone is skipped (spoken, Vosk heard "where is your mother" exactly)
Normalization (17–30)	asked = yes ("?"); where is your mother [mom → mother]
Reduction (32–43)	where is your mother [nothing removed]
Tokenize, Embedding (45–47)	[88, 41, 94, 50] → 4 × 64 numbers
Contextualize (49–51)	"your" looks 71% at "mother"; "mother" looks 52% at "where"
Space Restriction (53–54)	where kept, is → no sign, your → you dropped by the picker, mother kept; spoken order
Active Context (56–59)	remembers mother; topic neutral → whereabouts and plans
Display (61–63)	**where mother** (no punctuation)

Lines 1–10: how fast the recording stops. 8 sentences, each followed by silence:

Sentence	Speech ends	Recording stopped	Silence waited
"Where is your mother?"	1.02 s	2.10 s	1.08 s
"Do you need help with your class tomorrow?"	2.08 s	2.80 s	0.72 s
"What time is it?"	0.90 s	1.85 s	0.95 s
"I do not understand, can you say that again?"	2.94 s	3.80 s	0.86 s

Sentence	Speech ends	Recording stopped	Silence waited
“Hello, nice to meet you.”	1.82 s	2.60 s	0.78 s
“Are you going home now?”	1.26 s	2.10 s	0.84 s
“Is your teacher here today?”	1.44 s	2.35 s	0.91 s
“Why are you studying at school?”	1.74 s	2.60 s	0.86 s

Silence waited: 0.72–1.08 s, **0.88 s on average**. Recording took **2.5 s on average instead of the old fixed 5 s**. With only silence, it stops after 5 s (“nobody spoke”).

> **Remember**
>
> - Blocks follow the adviser's pipeline: Voice Recording → … → Display.
> - The microphone stops **about 0.9 s after the speaker stops**; at most 8 s; 5 s if nobody speaks.
> - **Reduction** removes unknown words (and a “not” that goes with one); **Space Restriction** removes words with no sign.
> - The Deaf reader sees **plain sign words**: no punctuation.

Algorithm 4 Space Restriction

In one sentence: Decide, for every word the hearing person said, whether it is shown to the Deaf person; the shown signs keep the order they were spoken in.

New in this version: rewritten after live testing. The signs keep their **spoken order** (the Hungarian slot assignment is gone), every **message sign is kept** (only my/you may be dropped), **"not" stays where it was said**, and no sign is shown twice in a row.

Algorithm 4: Space Restriction (RESTRICT TO SIGNS)

Input: *tagScores* (n × 17), *questionScores* (2), words $w_1 \ldots w_n$
Output: *glosses* (sign words in signing order)

```
1   Define OPTIONAL SIGNS ← {my, you}; // the only signs the data leaves out
2   isQuestion ← questionScores[1] > questionScores[0];
3   glosses ← [ ]; negated ← false;
4   for i ← 1 to n do // in the order the words were spoken
5       s ← normalizeWord(w_i);
6       if w_i ∈ NEGATIONS then
7           if NOT negated then append "not" to glosses; // where it was said
8           negated ← true;
9       else if s ∉ SIGN VOCABULARY OR w_i = "me" then
10          skip w_i; // no sign to show; "eat with me" is not MY
11      else if s = "fine" AND isQuestion then
12          skip w_i; // "is mother okay" → how mother
13      else if s ∈ OPTIONAL SIGNS AND
14              tagScores[i, 0] ≥ max(tagScores[i, 1…16]) then // drop score wins
15          skip w_i; // only my / you may be dropped by the picker
16      else if glosses is empty OR last of glosses ≠ s then
17          append s to glosses; // never the same sign twice in a row
18      end if
19  end for
20  if isQuestion then
21      glosses ← ["how"] + glosses;
22  end if
23  return glosses;
```

Line by line

Lines	What happens (plain words)	Why
1	OPTIONAL SIGNS = my, you.	The training data drops these two (about 100 and 95 times) and no other sign.
2	*isQuestion*: the question head says "add how" (a wellbeing question like "is he okay?").	
3–5	Go through the words **in the order they were spoken**; turn each into its sign name (your → you).	604 of the 608 training targets keep spoken order.
6–8	A negation (not, don't, no …) is shown once as "not", **where it was said**.	"sorry not understand", not "not sorry understand".
9–10	A word with no sign is skipped, and so is spoken "me".	"eat with me" is not MY (0 of 7 in the data).
11–12	In a "how" question, "fine" is skipped.	"is mother okay" → how mother.
13–15	my / you are skipped when the model's drop score wins.	The only signs left to the model.
16–17	Every other sign is added, unless it is the same as the sign just before it.	"nice to meet you, do you …" → one "you".
20–22	"how" goes first if it is a wellbeing question.	
23	Return the glosses.	

Try it: real runs through our code

"Good morning! Are you ready to study for class today?" P(add "how") = 0% → output **you study class today**

word	has a sign?	P(drop)	result	because
are	no	100%	removed	lines 9–10: no sign
you	yes	0%	**kept**	lines 16–17
to	no	100%	removed	lines 9–10: no sign
study	yes	0%	**kept**	lines 16–17
for	no	100%	removed	lines 9–10: no sign
class	yes	0%	**kept**	lines 16–17
today	yes	0%	**kept**	lines 16–17

"Today we will learn about the family." P(add "how") = 0% → output **today learn family**

word	has a sign?	P(drop)	result	because
today	yes	97%	**kept**	lines 16–17: a message sign is always kept (model said drop 97%)
we	no	100%	removed	lines 9–10: no sign
will	no	100%	removed	lines 9–10: no sign
learn	yes	0%	**kept**	lines 16–17
the	no	100%	removed	lines 9–10: no sign
family	yes	0%	**kept**	lines 16–17

"I do not understand, can you say that again?" P(add "how") = 0% → output **not understand again**

word	has a sign?	P(drop)	result	because
i	no	100%	removed	lines 9–10: no sign
do	no	100%	removed	lines 9–10: no sign
not	no	100%	"not", where it was said	lines 6–8
understand	yes	0%	**kept**	lines 16–17
can	no	99%	removed	lines 9–10: no sign
you	yes	100%	removed	lines 13–15: my/you, the model says drop (100%)
say	no	100%	removed	lines 9–10: no sign
that	no	100%	removed	lines 9–10: no sign
again	yes	0%	**kept**	lines 16–17

The live-test sentences, old rules vs new rules (the Deaf reader's screen):

The hearing person says	Old rules	New rules
"Good morning! Are you ready to study for class today?"	class you today study good	**you study class today**
"Today we will learn about the family."	family learn	**today learn family**
"Where are you? I am here at school now."	where school here you now	**where you here school now**
"Sorry, I do not understand. Can you say it again?"	not sorry again understand	**sorry not understand again**
"Do you want to eat with me?"	you eat my	**you eat**
"Please do not worry, it is fine."	not please fine	**please fine**
"Nice to meet you! Do you want to eat?"	nice you you eat	**nice you eat**
"I do not want water."	not water	**not water**

On the 608 training sentences, the new rules match the dataset 596 times (old: 601). The 6 differences keep the meaning: spoken order ("help please" instead of "please help", 4 times) and "yes" kept (2 times).

> **Remember**
> - A word reaches the screen only if it **has a sign**; every sign except my/you is **always kept**.
> - The signs keep the **order they were spoken in**; "not" stays where it was said.
> - The only word added is "how"; "not" replaces the spoken negation.

Algorithm 5 Active Context (memory and topic)

In one sentence: Remember who and what the conversation is about, and use it to fill in he/she (Case 2) or a missing person (Case 1).

New in this version: FILL SUBJECT adds a person **only when the signs are about a person** (lines 20–33), and **UPDATE TOPIC** (lines 41–63) follows the adviser's consultation: every context keeps a **weight**, words from other

contexts **leak in** instead of being set to 0, and the topic switches only when another context covers **at least half** of the turn. NOTE HEARING also counts **yes/no questions** as questions (line 12).

Algorithm 5: Active Context (shared conversation memory)

Input: context memory, the words or signs of the current turn
Output: updated glosses or signs, updated context memory

```
1   procedure RESOLVE PRONOUN(words, glosses, memory)
2       if memory.person ≠ none AND no gloss ∈ PEOPLE then
3           if ("he" ∈ words AND memory.person ∈ {father, teacher, friend}) OR
4              ("she" ∈ words AND memory.person ∈ {mother, teacher, friend}) then
5               insert memory.person after the leading question words;
6           end if
7       end if
8       return glosses;
9   end procedure
10  procedure NOTE HEARING(words, glosses, asked, memory)
11      memory.question words ← QUESTION WORDS in words ∪ glosses;
12      memory.last was question ← (memory.question words ≠ ∅) OR asked;
13      if "you" ∈ words then
14          memory.asked to you ← true;
15      else if some word ∈ PEOPLE then
16          memory.asked to you ← false;
17      end if
18      memory.person ← the last word ∈ PEOPLE (unchanged when none);
19  end procedure
20  procedure FILL SUBJECT(signs, memory)
21      if some sign ∈ PEOPLE OR "my" ∈ signs OR "you" ∈ signs OR
22         memory.person = none then
23          return (signs, none); // already says who, or nobody remembered
24      end if
25      aboutPerson ← (some sign ∈ QUESTION WORDS AND
26          every other sign ∈ STATES) OR
27          (some sign ∈ STATES AND no sign ∈ ACTIVITIES) OR
28          ("where" ∈ memory.question words AND every sign ∈ PLACES);
29      if aboutPerson AND NOT memory.asked to you then
30          return ([memory.person] + signs, memory.person);
31      end if
32      return (signs, none); // e.g. "where water" is about the water
33  end procedure
34  procedure NOTE DEAF(signs, sentence, memory)
35      memory.last was question ← false; memory.question words ← ∅;
36      if some word of signs or sentence ∈ PEOPLE then
37          memory.asked to you ← false;
38          memory.person ← the last such word;
39      end if
40  end procedure
41  procedure UPDATE TOPIC(words, repair, memory)
42      Define SWITCH SHARE ← 0.5, FADE ← 0.5;
43      C ← the signs of words that trigger some topic; // context words
44      if C is empty then
45          return; // nothing to measure: topic and weights stay
46      end if
47      T ← memory.topic; W ← memory.weights;
48      for each topic in TOPICS do
49          share[topic] ← |topic signs in C| / |C|; // weighted by confidence
50          W[topic] ← W[topic] + FADE × (share[topic] − W[topic]);
51      end for // a sign of T counts only for T; the others leak in
52      if repair then
53          T ← "help and repair"; // NOT + UNDERSTAND
54      else if T = none then
55          T ← the topic with the highest share;
56      else
57          best ← the other topic with the highest share; // tie: higher W
58          if share[best] ≥ SWITCH SHARE AND share[best] > share[T] then
59              T ← best; // a whole-context change, e.g., 9 of 16
60          end if
61      end if
62      memory.topic ← T;
63  end procedure
```

Line by line

Lines	What happens (plain words)	Why
`1–9`	**RESOLVE PRONOUN**: he/she → the remembered person, if it fits (he: father, teacher, friend; she: mother, teacher, friend).	"she" has no sign.
`10–19`	**NOTE HEARING**: remember the question words, whether it was a question at all (*asked*, yes/no questions too), whether it was asked to "you", and the person.	After "Is your mother here?", MOTHER HOME → "mother is home."
`20–24`	**FILL SUBJECT**: add nobody if a person, "my" or "you" was signed, or nobody is remembered.	The signs already say who.
`25–28`	The signs are **about a person** only if: a question with nothing else to ask about ("where"), OR how someone is ("fine"), OR a place answering "where is …?" ("home").	"where water" is about the water, not mother.
`29–32`	Add the person only then, and not if the last question was asked to "you".	Interviews: output only what was signed.
`34–40`	**NOTE DEAF**: the question is answered; remember the last person.	
`41–42`	**UPDATE TOPIC**. Two settings: SWITCH SHARE 0.5 and FADE 0.5.	See Part E.
`43–46`	Keep only the turn's **context words**, the signs that trigger some topic. If there are none, nothing changes.	"is", "you", "what" belong to no context.
`47–50`	Each context's **share** = its words ÷ all the context words (weighted by confidence). Every weight moves **halfway** to its share.	Words from other contexts **leak in**; a context nobody mentions fades ½, ¼, ⅛ …, never straight to 0.
`51`	A word that fits the active topic counts only for it.	"home" while talking about whereabouts stays whereabouts.
`52–53`	A repair always switches to help and repair.	NOT + UNDERSTAND.
`54–55`	Neutral (first turn): the context with the highest share starts the topic.	
`56–60`	Otherwise switch only if another context has **at least half** of the context words **and** more than the active one. A tie goes to the context with more weight from earlier turns.	Adviser: 9 of 16 words is a whole-context change; 2 of 7 only leaks.
`62`	Store the topic.	

Try it: real runs through our code

The 7 topics and their trigger signs (NOT, GO, SIGN, THANK YOU, SLEEP work once recorded):

Topic	Trigger signs
help and repair	help, please, understand, again, not*
school and learning	study, learn, teacher, class, school
whereabouts and plans	where, school, home, now, today, tomorrow, go*
greeting	hello, nice, friend, deaf, sign*
courtesy	sorry, please, thank you*
home and family	mother, father, family, home, eat, time, sleep*
feelings	fine, good

FILL SUBJECT (lines 20–33):

Said before	Signs	Screen	Active Context
"who is your teacher"	**what time class**	**what time is class?**	the question is about time, class, not a person
"where is your mother"	**where water**	**where is the water?**	the question is about water, not a person
"who is your teacher"	**why study**	**why are you studying?**	the question is about study, not a person
"where is your mother"	**how you today**	**how are you today?**	'you': the signs already say who
"is your mother okay"	**where**	**where is mother?**	added 'mother' from earlier
"where is your mom"	**home**	**mother is home.**	added 'mother' from earlier
"how is your father"	**fine**	**father is fine.**	added 'father' from earlier
"how is your father", "how are you"	**fine**	**i am fine.**	the last question was asked to the Deaf person

Leak or switch: the adviser's rule (talking about school, then one turn):

Active	Context words of the turn	Old rule	New rule	Why
school and learning	teacher, study, class, learn, school, mother, eat	school and learning	**school and learning**	stays school and learning (home and family 29% leaks in)
school and learning	teacher, study, mother, father, family, eat, time	school and learning	**home and family**	school and learning → home and family (71% of the context words)

A mixed conversation (Conversation_test.py), old rule vs new rule, with the weights after each turn:

Who	Turn	Old rule	New rule	Weights after (new)
says	"are you studying at school"	school and learning	**school and learning**	school and learning 0.50, whereabouts and plans 0.25
signs	"teacher study tomorrow"	school and learning	**school and learning**	school and learning 0.58, whereabouts and plans 0.29
signs	"teacher class mother"	school and learning	**school and learning**	school and learning 0.62, whereabouts and plans 0.15, home and family 0.17
signs	"teacher family eat home"	school and learning	**home and family**	school and learning 0.44, whereabouts and plans 0.20, home and family 0.46
says	"is your father at home"	home and family	**home and family**	school and learning 0.22, whereabouts and plans 0.10, home and family 0.73
signs	"not understand"	help and repair	**help and repair**	help and repair 0.50, school and learning 0.11, whereabouts and plans 0.05, home and family 0.36
signs	"please again"	help and repair	**help and repair**	help and repair 0.75, school and learning 0.06, whereabouts and plans 0.03, home and family 0.18

UPDATE TOPIC across one conversation (lines 41–63):

Who	Turn	Screen	Topic
says	"Hello, nice to meet you"	**hello nice**	topic neutral → greeting (100% of the context words)
signs	"hello nice friend"	**hello nice friend.**	topic stays greeting
says	"Where is your mother?"	**where mother**	topic greeting → whereabouts and plans (50% of the context words)
signs	"mother home"	**mother is home.**	topic stays whereabouts and plans (home and family 50% leaks in)
says	"What time is class?"	**what time class**	topic whereabouts and plans → home and family (50% of the context words)
signs	"teacher study tomorrow"	**teacher will study tomorrow.**	topic home and family → school and learning (67% of the context words)
signs	"not understand"	**i do not understand please say it again or write it.**	topic school and learning → help and repair (100% of the context words)
says	"Can you say that again?"	**again**	topic stays help and repair
signs	"help please mother"	**help please mother.**	topic stays help and repair (home and family 33% leaks in)
signs	"sorry"	**i am sorry.**	topic help and repair → courtesy (100% of the context words)

"What time is class?" is a 50/50 tie ("time" is a home-and-family word, "class" a school word); it goes to home and family because mother was just discussed, and the next school turn moves it to school.

> **Remember**
>
> - The memory holds the **person, question words, asked to you, topic**, and a **weight** per context.
> - A person is filled in **only when the signs are about a person**.
> - Topic: starts neutral; words from other contexts **leak** into the weights; it switches only at **half or more** of the turn; repair always wins; ties go to the context with more history.

Algorithm 6 Sentence Completion (TinyLlama), NEGATE and ADD PUNCTUATION

In one sentence: Write the English sentence one word at a time; if NOT was signed, put the negation into it; end it with "?" or "." for the hearing reader.

New in this version: the procedures **NEGATE** (lines 19–31) and **ADD PUNCTUATION** (lines 32–39).

Algorithm 6: Sentence Completion (GENERATE SENTENCE, NEGATE, ADD PUNCTUATION)

Input: *M* (embedded signs, k × 64); a sentence to negate or to punctuate
Output: *sentence* (English sentence)

```
1   procedure GENERATE SENTENCE(M)
2       y ← [<bos>]; // start-of-sentence token
3       for t ← 1 to MAX OUTPUT LENGTH do
4           H ← tinyLlama.embedToken(y);
5           for layer ← 1 to 2 do
6               H ← H + CausalSelfAttention(RMSNorm(H)); // earlier words only
7               H ← H + CrossAttention(RMSNorm(H), M); // reads the signs
8               H ← H + SwiGLU(RMSNorm(H));
9           end for
10          scores ← OutputHead(RMSNorm(H[last])); // 95 word scores
11          next ← argmax(scores); // greedy decoding
12          if next = <eos> then
13              break;
14          end if
15          append next to y;
16      end for
17      return tokenizer.decode(y without <bos>);
18  end procedure
19  procedure NEGATE(sentence) // a signed NOT is never lost
20      words ← split(toDeclarative(sentence));
21      if first word ∈ HELPER VERBS then // still a question: are you studying
22          insert "not" after the second word;
23      else if some word ∈ HELPER VERBS then // i am fine → i am not fine
24          insert "not" after the first helper verb;
25      else if first word ∈ {i, you, we} then // i want water → i do not want water
26          insert "do not" after the first word;
27      else
28          put "do not" in front; // help me → do not help me
29      end if
30      return join(words);
31  end procedure
32  procedure ADD PUNCTUATION(sentence) // the hearing reader's display only
33      if first word ∈ QUESTION STARTERS OR
34         some word ∈ QUESTION WORDS OR
35         a QUESTION STARTER is followed by you, i, he, she, it, we, they then
36          return sentence + "?"; // where is the water? / can you say it again?
37      end if
38      return sentence + "."; // mother is home.
39  end procedure
```

Line by line

Lines	What happens (plain words)	Why
1–2	**GENERATE SENTENCE**: start with the <bos> (beginning) marker.	
3	Repeat at most 16 times, one word each time.	
4–9	Two decoder blocks: look back at earlier words (6), look at the signs *M* (7), refine (8).	Line 7 is how the signs steer the sentence.
10–11	Score all 95 words; take the highest (greedy).	
12–15	Stop at <eos>; otherwise add the word.	
17	IDs → words.	
19–20	**NEGATE**: first turn a question form into a statement.	"is mother home" → "mother is home".

Lines	What happens (plain words)	Why
21–22	Still a question (starts with is/are/can …): put “not” after the second word.	“are you not studying”.
23–24	Otherwise put “not” after the first helper verb (am, is, are, will, can, do …).	“i am not fine”.
25–26	No helper verb, starts with i/you/we: add “do not”.	“i do not want water”.
27–28	Otherwise “do not” in front.	“do not help me”.
32–35	**ADD PUNCTUATION**: a question if it starts with a question starter (is, are, can, what …), contains a question word (what, where, why, how), or has a starter followed by a subject (can you, are you …).	“sorry can you say it again?”
36–38	Question → “?”; otherwise “.”.	Only the hearing reader's screen is punctuated.

Try it: real runs through our code

GENERATE SENTENCE: signs mother (from memory) + home:

Step	Written so far	Top 3 next words	Line 7 looks at	Chosen
1	(start)	is 100% · this 0% · i 0%	mother 66% · home 34%	**is**
2	is	mother 100% · teacher 0% · father 0%	mother 100% · home 0%	**mother**
3	is mother	home 100% · at 0% · fine 0%	mother 21% · home 79%	**home**
4	is mother home	<eos> 100% · tomorrow 0% · now 0%	mother 11% · home 89%	**<eos>**

Result “is mother home” → Algorithm 2 lines 36–38 → “mother is home” → ADD PUNCTUATION → **mother is home.**

NEGATE on its own:

Sentence	After NEGATE
“are you studying”	**are you not studying**
“i am fine”	**i am not fine**
“is mother home”	**mother is not home**
“i want water”	**i do not want water**
“help me”	**do not help me**

ADD PUNCTUATION and yes/no questions, old rules vs new rules (the hearing reader's screen):

Said before	Signs	Old rules	New rules
	where water	where is the water	**where is the water?**
“Is your mother here?” →	**mother home**	is mother home	**mother is home.**
	sorry again	sorry can you say it again	**sorry can you say it again?**
	eat now	i am eating now	**i am eating now.**

> **Remember**
> - One word per loop; stops at <eos> or after 16 words.
> - NEGATE never loses a signed NOT: after a helper verb, or “do not”.
> - ADD PUNCTUATION: “?” for a question, “.” otherwise; the Deaf reader's screen has none.

Part D · Follow the lines: one conversation through every algorithm

Turn 1: the hearing person types **“Where is your mom?”**. Turn 2: the Deaf person signs **“home”**. Read top to bottom: this is the order in which the lines run.

Algorithm	Lines	What happens
Alg 1	`18 → 24 → 25 → 27`	mode 2, typed “Where is your mom?”, call Algorithm 3
Alg 3	`1 → 13–14`	typed, so the microphone is skipped
Alg 3	`17–30`	asked = yes; where is your mother (mom → mother)
Alg 3	`32–40 → 45 → 47`	nothing removed; IDs [88, 41, 94, 50]; 4 × 64 numbers
Alg 3	`49–51 → 53–54 → Alg 4`	where, mother kept in spoken order; is (no sign) and your (my/you) dropped
Alg 3	`56 → Alg 5 lines 1–9`	no he/she: nothing changed
Alg 3	`57 → Alg 5 lines 10–19`	memory: person = mother, question words = where, a question
Alg 3	`58–59 → Alg 5 lines 41–63`	no repair; topic neutral → whereabouts and plans
Alg 3	`61–63 → Alg 1 line 28`	**where mother** on the deaf-mute individual's screen (no punctuation)
Alg 1	`18 → 19 → 20 → 22`	mode 1, sign “home” (confidence 1.0), call Algorithm 2
Alg 2	`1–12 → 13–17`	home kept (sure enough, not a negation); not empty
Alg 2	`19 → 21`	home → ID 36 → 1 × 64 numbers
Alg 2	`23 → 24 → Alg 5 lines 41–63`	no repair; “home” is a whereabouts trigger, so the topic stays
Alg 2	`25–26 → Alg 5 lines 20–33`	a place answering “where” → about a person → add mother
Alg 2	`27–29`	M = mother + home (2 × 64)
Alg 2	`35 → Alg 6 lines 1–18`	TinyLlama writes “is mother home”
Alg 2	`36–38`	last turn was a question → “mother is home”
Alg 2	`39–44`	both signs present, no NOT → unchanged
Alg 2	`46 → Alg 5 lines 34–40 → 48 → Alg 6 lines 32–39`	a statement → ends with “.”
Alg 1	`23`	**mother is home.** on the non-signer's screen

Part E · Where each number comes from

For each number or rule: where it is in the algorithm, its basis, and how strong the evidence is.

Number / rule	Where	Basis	Evidence
16,000 Hz	`Alg 1 line 14; Alg 3 line 5`	The speech model's own file `conf/mfcc.conf`: `--sample-frequency=16000`; it analyses 20–7,600 Hz, and 16 kHz sampling captures up to 8,000 Hz (Nyquist).	Solid: model file
≈0.9 s silence ends a recording	`Alg 3 line 6`	Model file `conf/model.conf`: end-of-speech rules 0.5 / 0.75 / 1.0 s of silence. Measured on 8 sentences: 0.72–1.08 s, mean 0.88 s.	Solid: model file + measurement
5 s with no speech	`Alg 1 line 15; Alg 3 line 9`	Matches the model's built-in no-speech rule: on 12 s of pure silence, Vosk itself declared the end at 5.2 s.	Solid: measured
8 s maximum	`Alg 1 line 15; Alg 3 line 8`	16 words ÷ about 2.5 words per second (≈150 words per minute) ≈ 6.4 s, + 0.9 s end detection ≈ 7.3 s → 8 s.	Derived: confirm with real speakers
2.5 s instead of 5 s	`Alg 3 lines 1–10`	Average recording length on the 8 test sentences.	Measured (synthetic voice)
16 words	`Alg 1 lines 11–12`	Both models have 16 word positions.	Model design
0.6 confidence	`Alg 1 line 13; Alg 2 line 4`	Placeholder. To get the real number: run the recognizer on validation clips (signs + non-vocabulary gestures), and pick the lowest threshold where the kept signs are, e.g., ≥ 95% correct.	**Not yet evidenced**
7 topics + triggers	`Alg 1 line 10; Alg 5 lines 41–63`	Interviews: every topic in 4 or 5 of 5 groups.	Interview data
Switch at half (0.5)	`Alg 5 lines 42, 58`	Adviser's rule: a whole-context change is a majority of the turn (his example: 9 of 16 words). Test: 5 of 7 home-and-family words switches; 2 of 7 only leaks; topic still correct in 34/34 dialogue turns and 7/7 mixed turns.	Adviser + test
Fade 0.5 per turn	`Alg 5 lines 42, 50`	Design value: a context nobody mentions keeps ½, ¼, ⅛ of its weight after 1, 2, 3 turns, so it fades instead of dropping to 0. Confirm with the school conversation tests.	**Design value**: confirm in tests
Repair rule	`Alg 2 lines 23, 32–33`	Interviews: "do not understand" in 4 of 5 groups; High priority.	Interview data
Keep NOT	`Alg 2 lines 6–7, 41–43`	Interviews: NOT in 5 of 5 groups. Our test: before, NOT UNDERSTAND → "i understand".	Interview + test
Spoken order	`Alg 4 lines 3–5`	604 of the 608 training targets keep the spoken order. Live test: the picker's positions turned "Are you ready to study for class today?" into "class you today study good".	Training data + test
Only my / you optional	`Alg 4 lines 1, 13–15`	Training data: the only signs the targets leave out are my (100 times) and you (95 times). The picker had dropped "today" (97% drop score) from "Today we will learn about the family".	Training data + test
"me" not shown as MY	`Alg 4 lines 9–10`	Training data: 0 of the 7 sentences with "me" show MY.	Training data
596 / 608 training sentences	`Alg 4`	After the output fixes (was 601). The 6 differences keep the meaning: spoken order (4) and "yes" kept (2). Conversation test unchanged: topic 34/34, mixed contexts 7/7.	Measured
Fill a person only when about a person	`Alg 5 lines 20–33`	Interviews: "output only what was recognized". Our test: 4 wrong outputs fixed, 6 correct ones kept.	Interview + test

The recording measurements used the Windows synthetic voice, which speaks faster and more clearly than a person in a classroom; measure real speakers during the school tests to confirm them.

Cheat sheet

The numbers

34 / 57 / 91 / 95	signs / helper words / lookup table / token IDs
16	max words in and out
0.6	confidence threshold (placeholder)
16,000 Hz	microphone sample rate
≈0.9 s	silence that ends a recording
5 s / 8 s	give up if nobody speaks / longest turn
7	topics from the interviews
0.5 / 0.5	topic switch share / fade per turn
64	numbers per word
17	picker scores per word: drop, or slots 1–16 (now only keep vs drop is used)
604 / 608	training targets in spoken order
75,411 / 133,376	learned numbers: word picker / TinyLlama

Each algorithm in one line

1 · Main Program	set up once, then route every turn and show the text
2 · Sign-to-Sentence	signs → English sentence; unsure signs dropped; NOT kept; repair rule
3 · Speech-to-Gloss	speech → sign words; recording stops when the speaker stops
4 · Space Restriction	only signs, in spoken order; only my/you may drop; "not" where said; add how
5 · Active Context	memory + topic weights; other contexts leak, switch at half; a person only when about a person
6 · Sentence Completion	write the sentence word by word; NEGATE; ADD PUNCTUATION (hearing reader only)

The two outputs

Case 1 → an English **sentence** on the screen facing the non-signer.
Case 2 → **sign words** on the screen facing the deaf-mute individual.

Things people mix up

- **Reduction** removes unknown words; **Space Restriction** removes words with no sign.
- **Dropped** (unsure sign, Case 1) is not the same as **removed** (no sign, Case 2).
- Case 1 embedding has **no position**; Case 2 has **position**.
- The memory adds a person only when the signs are **about a person**.
- 0.6 is a **placeholder**; 16 kHz, 0.9 s and 5 s have evidence.

Self-test

Answer without looking; the answers are on the next page.

1. Which lines of Algorithm 1 run only once?
2. What decides whether Algorithm 2 or Algorithm 3 runs?
3. A sign arrives with confidence 0.4. What happens?
4. Why is NOT kept as a flag and not as a sign?
5. What does NOT + UNDERSTAND produce?
6. “not fine” right after “How is your father?” gives what?
7. When does FILL SUBJECT add a person?
8. Why doesn't “where water” become “where is mother …”?
9. When does the topic switch?
10. What does it mean that a context “leaks”?
11. “help please mother” during a help request: topic?
12. Why does the microphone record at 16,000 Hz?
13. When does the recording stop after someone speaks?
14. What if nobody speaks?
15. Why are “is” and “your” not removed by Reduction?
16. Why does Case 2 add position to the embedding but Case 1 does not?
17. Why are the signs shown in the order they were spoken?
18. Which signs may the picker drop?
19. Why is there no punctuation on the Deaf reader's screen?
20. What does NEGATE do with “i want water”?
21. Which number still has no evidence?
22. What does RESET do?

Self-test answers

1. **Which lines of Algorithm 1 run only once?**
   Lines 1–15 (Initialize).
2. **What decides whether Algorithm 2 or Algorithm 3 runs?**
   The mode, Algorithm 1 line 18.
3. **A sign arrives with confidence 0.4. What happens?**
   Algorithm 2 lines 4–5: it is skipped (below 0.6).
4. **Why is NOT kept as a flag and not as a sign?**
   NOT is not one of the 34 signs; dropping it would flip the meaning (Algorithm 2 lines 6–7).
5. **What does NOT + UNDERSTAND produce?**
   The repair sentence "i do not understand please say it again or write it" (Algorithm 2 lines 23, 32–33), and the topic becomes help and repair.
6. **"not fine" right after "How is your father?" gives what?**
   "father is not fine": FILL SUBJECT adds father, NEGATE adds not.
7. **When does FILL SUBJECT add a person?**
   Only when the signs are about a person (a question with nothing else, how someone is, or a place answering "where"), no person/my/you was signed, and the last question was not asked to "you" (Algorithm 5 lines 20–33).
8. **Why doesn't "where water" become "where is mother ..."?**
   The question is about the water, not a person (Algorithm 5 lines 25–32).
9. **When does the topic switch?**
   Repair always switches to help and repair; otherwise only when another context covers at least half of the turn's context words and more than the active one (Algorithm 5 lines 56–60).
10. **What does it mean that a context "leaks"?**
    A word from another context raises that context's weight without switching the topic; the weight then fades by half each turn (Algorithm 5 lines 47–51).
11. **"help please mother" during a help request: topic?**
    Stays help and repair: only 1 of 3 context words is home and family, so it leaks in (33%).
12. **Why does the microphone record at 16,000 Hz?**
    The speech model was built for it (its file conf/mfcc.conf).
13. **When does the recording stop after someone speaks?**
    About 0.9 s after they stop (Algorithm 3 line 6), measured 0.72–1.08 s.
14. **What if nobody speaks?**
    It stops after 5 s (Algorithm 3 line 9).
15. **Why are "is" and "your" not removed by Reduction?**
    They are helper words in the lookup table; Space Restriction removes them later.
16. **Why does Case 2 add position to the embedding but Case 1 does not?**
    English word order carries meaning; the order signs arrive in does not.
17. **Why are the signs shown in the order they were spoken?**
    604 of the 608 training targets use that order, and the picker's position scores scrambled new sentences (Algorithm 4 lines 3–5).
18. **Which signs may the picker drop?**
    Only my and you, the two the training data leaves out; every other sign is kept (Algorithm 4 lines 13–17).
19. **Why is there no punctuation on the Deaf reader's screen?**
    It shows plain sign words; only the hearing reader gets "?" or "." (Algorithm 6 lines 32–39).
20. **What does NEGATE do with "i want water"?**
    "i do not want water" (Algorithm 6 lines 25–26).
21. **Which number still has no evidence?**
    The 0.6 confidence threshold; it must be calibrated on the recognizer's data. The 0.5 fade is a design value to confirm in tests.
22. **What does RESET do?**
    Empties the memory, including the topic (Algorithm 1 lines 29–30).